

Evaluación 2 - Taller de Programación

Problema del camino máximo en un grafo

Prof. Pablo Román, Miguel Cárcamo

2026/04/29

1 Objetivos

- Desarrollar competencias en la optimización puramente combinatoria en grafos: El estudiante deberá aprender el problema del camino máximo, de complejidad NP-Hard, y entender por qué encontrar soluciones para grafos grandes es un problema difícil.
- Diseño heurístico de algoritmos: Aprender a diseñar e implementar algoritmos heurísticos (e.g. local search) para encontrar soluciones de alta calidad en un tiempo razonable. Se dará énfasis al trade-off entre soluciones de alta calidad y el tiempo de ejecución, dado que un algoritmo exhaustivo resulta impráctico para un problema NP-hard. Esta tarea se centra en el concepto de **Local Search**.
- Mejores prácticas de desarrollo en C++: Desarrollar un proyecto individual en C++ con enfoque en la programación modular. Deberá crear clases para la representación de grafos y de algoritmos, usar librerías de contenedores de STL y mantener una estructura ordenada del código fuente con compilación separada.
- Desarrollo Experimental: Evaluar algoritmos en conjuntos de datos y analizar su rendimiento. Se ejecutarán experimentos en grafos clasificados en fáciles, medianos y difíciles, midiendo la calidad de la solución frente a su tiempo de ejecución. Mediante esta experimentación, se podrán ajustar algoritmos adecuados para resolver el problema planteado.

Al finalizar este proyecto, los alumnos adquirirán experiencia en tratar con problemas NP-Hard utilizando heurísticas, desarrollando en código C++ de forma estructurada e interpretando resultados experimentales.

2 Metáfora: La guarida del dragón

Muchos años atrás, Bilbo logró ingresar a la guarida del dragón y traerse un gran tesoro en su mochila. Tuvo que seleccionar qué piezas poner en la mochila que maximizaran el valor total, pero sujeto a un peso máximo para lograr arrancarse del dragón. Balin, un amigo suyo, decidió volver pero con un camión para llevarse los tesoros. Después del desastre que causó Bilbo, toda la guarida quedó conformada por túneles interconectados sumamente



Figure 1: Un tunel

inestables, pero conteniendo grandes tesoros. El dragón ya no está, pero se suma el peligro de derrumbe. El camión de Balín vuelve inestable por donde pasa y hace que se derrumbe la entrada, salida y todo el túnel. Sin embargo, por cada túnel k se obtiene un beneficio p_k pero también un peso de w_k . El máximo peso que puede llevar el camión es W . La avaricia de Balín no tiene comparación, ya que está dispuesto a derrumbar todos los túneles posibles de forma de tener el máximo beneficio. El problema de Balín es: ¿por cuáles túneles pasar con tal de obtener el máximo beneficio? La red de túneles puede representarse como un grafo. Cada arista del grafo es un túnel, los nodos representan interconexiones entre túneles. En este grafo hay un nodo de entrada $i = 0$ y un nodo de salida $i = N$. El hecho de que se derrumben los túneles, hace que un camino factible del problema debe: partir en el nodo de inicio ($i = 0$), llegar al nodo final ($i = N$), entremedio no puede repetir nodos y no puede sobrepasar la capacidad del camión W . El camino con mayor beneficio es el que se busca.

3 Formulación matemática del problema

Un grafo $G = (V, E)$ compuesto de vértices o nodos (V) y aristas E . Cada arista $k \in E$ une dos nodos $i, j \in V$ ($k = (i, j)$). Se conviene que el nodo $i = 0$ es la entrada y $i = N$ es la salida. Consideramos que tenemos $N + 1$ vertices y $M + 1$ aristas. Existe un beneficio p_k que se puede obtener al atravesar la arista k pero tiene un costo o peso w_k por hacerlo. Un camino C de $Q + 1$ pasos se puede representar como una secuencia de aristas $C = [k_0, \dots, k_Q]$, donde la arista k_0 esta conectada a la entrada y la arista k_Q a la salida. El problema de optimización a resolver es el siguiente:

$$\begin{aligned}
& \max_C \sum_{k \in C} p_k & (1) \\
& \text{s.a.} \\
& \sum_{k \in C} w_k \leq W \\
& k_0 = (0, j) \\
& k_Q = (i, N) \\
& k_l = (i, j) \wedge k_{l+1} = (i', j') \implies j = i' \\
& C = [k_0, k_1, \dots, k_Q], k_l \in E, \text{NOLOOP}
\end{aligned}$$

La primera restricción es el peso máximo. La segunda y tercera restricción es que inicie y termine correctamente, la cuarta indica que los tuneles escogidos se encuentren interconectados, finalmente el camino C se indica como una secuencia de aristas de cualquier largo pero sin repetir y sin generar ciclos.

Otra representación para el camino, es considerar la secuencia de nodos $C = [0, v_1, \dots, v_{Q+1}, N]$. Esta representación tiene la ventaja que se indica de manera explícita la entrada (0) y salida (N), y que la condición de no tener ciclos implica no repetir nodos. Desde el punto de vista de la implementación conviene disponer de ambas representaciones de modo de simplificar el chequeo de las condiciones. Esta representación, permite estimar el orden de cómputo en el peor caso en un grafo denso. La cantidad de caminos posibles para un camino con $Q + 2$ nodos es $(N - 1)! / (N - 1 - Q)!$ debido a las permutaciones de nodos intermedios. Todos los caminos posibles se obtiene sumando sobre Q queda proporcional a $(N - 1)! = O(N^N)$.

4 The Orienteering Problem

Este problema se denomina en la jerga de optimización como "Orienteering Problem" (<https://www.sciencedirect.com/science/article/pii/S037722171630296X>), su traducción al castellano sería problema de orientación. A pesar del nombre, este problema encuentra el camino de máximo beneficio con restricción de capacidad. Este problema se considera como una combinación del vendedor viajero y el problema de la mochila. Se ha visto que tiene distintas aplicaciones:

- logística y en ruteo de vehículos (<https://www.sciencedirect.com/science/article/abs/pii/S0377221725000347>)
- Planificación de viajes (<https://open.metu.edu.tr/handle/11511/113098>)
- Robótica y plan de vuelo de drones (<https://ui.adsabs.harvard.edu/abs/2014arXiv1402.1896Y/abstract>)

- Redes de sensores y movimientos de robots (http://biorobotics.ri.cmu.edu/papers/paperUploads/ICRA2020_Visual_Coverage.pdf)
- Respuesta a desastres y planes de evacuación (https://www.researchgate.net/publication/317025120_An_ienteering-based_approach_to_manage_emergency_situation).

El problema es NP-hard, es decir no es resoluble en una cantidad polinomial de operaciones. En principio, si tuviésemos que contar cuántos caminos existen entre la entrada y salida considerando $|E| = M$ aristas es de un orden exponencial. Por lo tanto, listar todos los caminos es imposible para un gran número de aristas. La solución a este problema no puede ser realizado por fuerza bruta y tiene que aproximarse mediante alguna heurística. Se han estudiado varios algoritmos que encuentran un valor menor que el óptimo, pero al menos cercano a el y que termina en un tiempo razonable.

5 Algoritmos y heurísticas

Nuestro enfoque será utilizar métodos aproximados y heurísticos para encontrar una solución no óptima, pero cercana a la solución esperada. Partiremos describiendo algoritmos exactos que, en el peor de los casos, revisan exhaustivamente toda la combinatoria posible. No son heurísticos, pero pueden conformar la base para dichos algoritmos de aproximación.

5.1 Algoritmos exactos

Estos algoritmos encuentran la solución óptima ya que buscan en todas las combinaciones. Sin embargo, si $N, M \sim 100$ se vuelven inutilizables. Para casos del orden de $N, M \sim 10$, es el mejor algoritmo. Otra observación es de alguna forma terminar el algoritmo en el mejor camino obtenido hasta el momento, esto evita revisar todos los casos pero ya no se tendrá el óptimo sino sólo una aproximación.

- **Fuerza bruta:** Es lo más sencillo de realizar. Se itera sobre todos los casos, manteniendo el máximo contabilizado hasta el momento. Para revisar todos los casos, es necesario codificar un camino $C = [k_0, \dots, k_Q]$ puede ser considerado como una lista de aristas o una lista de vértices $C = [0, v_1, \dots, v_Q, N]$. Se puede realizar un mecanismo de backtracking que itera sobre todos los vecinos no visitados del último vértice y sobre todos ellos se vuelve a buscar. Entre todos los caminos generados se busca el que tenga el mayor beneficio y que cumpla la restricción de peso máximo. Después de revisarlos todos se entrega el camino máximo. Siempre se puede podar durante el proceso, pero esto lo dejamos como una mejora para el siguiente punto.
- **Ramificación y acotamiento:** Es una mejora respecto al caso anterior, ya que permite evitar la revisión de ciertos casos, un proceso denominado poda (ver https://en.wikipedia.org/wiki/Branch_and_bound). El algoritmo de Branch and Bound se basa en estados que representan soluciones parciales al problema, es decir caminos parciales. Siguiendo con la notación anterior en el caso de representar un camino como lista de vértices $C = [0, v_1, \dots, v_Q, N]$, se genera un estado que representa el camino hasta un cierto vértice l es decir $C = [0, v_1, \dots, v_l]$. Este camino incompleto se ramifica en los siguientes descontando aquellos que repiten nodos y que sobrepasan peso máximo. Se puede podar aún más, para ello se mantiene una cota superior y otra inferior. La cota superior no es la solución, pero indica cuán grande podría ser la función objetivo en general exagerando. La cota inferior es calculada utilizando un algoritmo más rápido, por ejemplo goloso u otra heurística, por eso es inferior porque no llega al óptimo (mayor ganancia). Dichas cotas inferior y superior coinciden en el caso de tener un valor óptimo. Estos estados se pueden mantener ordenados en un Heap según su cota inferior. Al revisar el nodo con mayor cota inferior, se seleccionan uno o más vértices y se ramifican en diferentes estados nuevos. El algoritmo mantiene el valor máximo de cut de todas las cotas inferiores que es el primero en el heap. Esto constituiría un criterio de poda. Si una nueva ramificación tiene una cota superior menor que la mayor cota inferior actual, se puede desestimar la búsqueda a partir de dicho estado. La descripción anterior no especifica cuál es el criterio de cálculo de las cotas superiores e inferiores. Esto provoca que este proceso sea modificable y mejorable. Mientras más ajustadas sean las cotas a utilizar, mejor será el algoritmo propuesto, debido a una posible mayor cantidad de poda.

Sin embargo, la utilidad de los algoritmos exactos se ve claramente diezmada a medida que aumenta el número de vértices. Estos algoritmos, en el peor de los casos, no realizan ninguna poda y revisan todos los casos posibles. En el caso $n \geq 10$ se utilizan en general algoritmos heurísticos. Es importante recalcar que los algoritmos exactos siguen siendo útiles para grafos pequeños, ya que pueden encontrar soluciones exactas en subgrafos de pequeño tamaño.

A pesar de lo anterior si se puede utilizar Branch and Bound limitando la cantidad de iteraciones y retornando la mejor cota inferior que se tenga hasta el momento. Esto implica que renunciando a obtener el optimo el esquema anterior, Branch and Bound se puede utilizar como herramienta para orquestar algoritmos más sofisticados. Revisemos los siguientes detalles de implementación que pueden mejorar el algoritmo de Branch and Bound:

- **Poda utilizando camino mínimo en peso D_v desde v :** Se busca el costo en peso $D_v(C)$ mediante el algoritmo de Dijkstra el camino más corto desde el nodo v a la salida en términos de peso, sin repetir vértices ya utilizados en el camino parcial C y siendo v un vecino del último vértice l del camino $C = [0, v_1, \dots, v_l]$. Este número indica el menor peso a sumar al peso actual que lleva el camión, su importancia radica en que permite obtener factibilidad de seguir el camino por dicho nodo. Si no es posible, entonces se descarta (poda). Entonces la condición de poda por seleccionar el vértice v es:

$$\left(\sum_{k \in C} w_k\right) + w_{(v_l, v)} + D_v(C) > W, v \in \text{Vecino}_C(v_l) \quad (2)$$

Esto tiene la desventaja de calcular Dijkstra en cada ramificación. Sin embargo, se puede dejar precalculado el costo en peso D_v desde cada vértice $v \in V$ utilizando Dijkstra de forma independiente del camino previo. Este costo cumple con $D_v \leq D_v(C)$, el costo sólo puede bajar si no se considera la restricción del camino. Entonces se calcula el costo en peso mediante Dijkstra desde cada vértice a la salida D_v . Para ese caso, la condición menos costosa para podar consiste en:

$$\left(\sum_{k \in C} w_k\right) + w_{(v_l, v)} + D_v > W, v \in \text{Vecino}_C(v_l) \quad (3)$$

Donde $\text{Vecino}_C(v)$ son todos los vecinos de v no contenidos en el camino C . Esto indica que si se sobrepasa el límite de peso por usar v entonces no hay forma que se reduzca por cualquier otro camino. Aunque la primera condición es más ajustada que la segunda, es también más cara en tiempo de cómputo. El hecho que sea más ajustada permite una mayor probabilidad de poda. Pueden usarse ambas. Por ejemplo, dosificar el uso de la poda mas cara cada cierto tiempo.

- **Cota inferior del beneficio:** Recordemos este problema trata sobre encontrar el camino con mayor beneficio, dado esto cualquier camino tiene beneficio menor. Para obtener una cota inferior solo debemos encontrar un camino y calcular su beneficio, pero debe cumplir con la restricción de peso. Una forma rápida de encontrar un camino cualesquiera es utilizar Dijkstra para calcular el camino mas corto en peso total, se debe modificar un poco por la restricción de peso. Eso entregará una cota inferior, pero mientras más alejada del camino de mayor beneficio menor será el aporte del algoritmo de branch and bound. Como los estados se ordenan de mayor a menor cota inferior, si se selecciona el mayor se espera que ese número indique mayor probabilidad de encontrar el óptimo. Se debe utilizar un mejor camino y para ello se mencionan varios algoritmos heurísticos mas adelante. Si utilizamos un buen algoritmo Heurístico en esta parte, entonces Branch and Bound será mejor que dicho algoritmo incluso si se limita el número de iteraciones.
- **Cota superior del beneficio:** Se requiere una sobre-estimación del beneficio, pero lo más cercana posible al óptimo. Esto se puede lograr desestimando restricciones y contabilizando los beneficios de las aristas que se agregarían. En este problema hay dos restricciones, una es la estructura de caminos del grafo que incluyen varias subrestricciones como que sea un camino, que no se repitan nodos y otras; y la segunda es la restricción de máximo peso. Por ejemplo si desestimamos las dos restricciones una cota superior UB es considerar que se va a pasar por todas las aristas restantes a nodos distintos de los usados que llamamos conjunto C^c :

$$UB = \sum_{k \in C} p_k + \sum_{k \in C^c} p_k \quad (4)$$

El conjunto C^c en general no va a conformar la continuación del camino de manera válida y tampoco se pide que cuide la restricción de peso. Corresponde a una sobre-estimación y puede servir como cota superior. Sin embargo es muy alejada del óptimo lo que significa que no va a discriminar muy bien entre estados para poder realizar la poda. Otra idea, es mantener la restricción de peso pero no de camino. En ese caso, se pueden ordenar las aristas que faltan por mayor beneficio, cuidando que no sean incidentes a los nodos actualmente utilizados en C . De manera golosa se agregan los beneficios de cada arista hasta completar la capacidad. En dicho caso, esta cota será menor que la anterior, pero sigue siendo mayor que el optimo. Esto se refina aún mas si el orden es por el valor p_k/w_k y el ultimo valor que hace que se sobrepase el peso máximo se agregue fraccionariamente.

5.2 Algoritmos Heurísticos

Estas estrategias no garantizan de ninguna forma encontrar el óptimo absoluto, pero sí se ejecutan en tiempos relativamente cortos. Sin embargo, la experiencia muestra que en algunos casos se logra aproximar adecuadamente. Esta tarea requiere que usted experimente con dichos métodos para encontrar la mejor solución posible.

- **Algoritmo goloso simple:** Un algoritmo avaro es una heurística que construye la solución en pequeños pasos, maximizando el valor de la función objetivo. En este caso, se genera un recorrido en profundidad mientras queden vértices por seleccionar, se escoge la arista vecina al último vértice v del camino actual $k = (i, j)$ con i y j vértices no seleccionados, esta arista es la con mayor valor de p_k/w_k para colocar k como posible próximo vértice en el camino. Si el recorrido en profundidad no llega a la salida se procede con el siguiente vecino más conveniente. La primera búsqueda en profundidad que termine entrega en camino.
- **Local Search:** Más que un algoritmo, es un concepto derivado de la optimización continua, que ejecuta pasos en la dirección de máximo crecimiento. Esta heurística se basa en modificar un camino que cumpla con las restricciones y se vaya modificando mediante pequeños ajustes. Se selecciona una configuración de partida C_0 , por ejemplo, utilizando el algoritmo goloso. Se denomina búsqueda local porque modifica el vector C_l mediante operaciones predefinidas, aceptando los cambios mientras el valor de la función objetivo, $cut(k)$, crezca. El algoritmo termina cuando ya no hay operaciones que permitan incrementar la función objetivo o se llega a un máximo de iteraciones. Todo depende de las operaciones que se deciden utilizar. A continuación, detallamos algunas estrategias de Local Search.
 - **2-OPT Search:** Este corresponde a un algoritmo de “Local Search”, que utiliza un conjunto de operaciones 2-OPT. La operación 2-OPT intercambia 2 vertices en el camino de C_l si es que existen los correspondientes vértices en el grafo. Por lo tanto, son $n(n - 1)/2$ operaciones, una por cada par de vértices. La aplicación de las operaciones puede realizarse de forma aleatoria, lo que ha demostrado mejores resultados en general. Se ha observado que este tipo de búsqueda se queda atrapado en óptimos locales. Sin embargo, es útil y rápida para encontrar una aproximación al óptimo actual.
 - **K-OPT Local Search:** Este caso generalizado de 2-OPT. Cada operación considera permutaciones de $K \leq n$ vértices en C_l . En este caso, la cantidad de operaciones posibles son todas permutaciones de N en K . Es una mayor cantidad de operaciones, lo cual permite explorar más ampliamente el espacio de todas las combinaciones, porque permite alcanzar una distancia mayor (K pasos de edición).
 - **Breakout Search:** El fenómeno de quedar atrapado en mínimos locales puede evitarse mediante el mecanismo de Fuga (Breakout) (<https://doi.org/10.1016/j.engappai.2012.09.001>). La idea es explorar efectuando un gran salto con K-OPT, aunque se empeore la función objetivo; posteriormente, refinar usando la búsqueda local con 2-OPT, si se llega a un mínimo local menor, se acepta y se prosigue (<https://leria-info.univ-angers.fr/~jinkao.hao/papers/BenlicHaoMaxCut2012.pdf>). Se mantiene una lista de mínimos locales para evitar generarlos de nuevo.
 - **Scatter Search:** Un algoritmo de Scatter Search construye una nueva solución mediante la combinación de dos o más soluciones (ver <https://doi.org/10.1287/ijoc.1080.0275>). Se mantiene un conjunto de caminos con el que se lleva a cabo este proceso. Un proceso básico de combinación de soluciones entre dos vectores consistiría en variar ambos vectores mediante pequeños cambios que transforman un vector en otro, por ejemplo, mediante una operación 2-OPT. A partir de soluciones intermedias, se realiza una búsqueda local para comprobar si los mínimos locales obtienen valores inferiores a los anteriores. Esto actualiza el conjunto de soluciones y elimina las peores. Se repite el proceso hasta no obtener una mejora.
 - **Greedy Randomized Adaptive Search Procedure (GRASP):** Este método es muy similar a los anteriores (ver <https://www.scielo.cl/pdf/ingeniare/v16n3/art05.pdf>). Se mantiene un conjunto de soluciones. Se selecciona uno al azar y se realiza una búsqueda local de mejora. Como en el caso anterior, se generan soluciones adicionales mediante combinaciones de dos.

6 Implementación

Se exige que la implementación sea eficiente y escrita de manera limpia y clara, en el lenguaje de programación C++. Para esta tarea se deben utilizar los contenedores y los algoritmos de la librería STL. Se debe cuidar la orientación al objeto: No deben haber funciones sueltas, solo clases y funciones main para tests y programa principal. Es importante que el programa final resuelva el problema de forma eficiente. Cada clase debe tener asociado un programa de test.

El programa principal (main.cpp) deberá mostrar un menú. Este menú también incluye las siguientes opciones:

1. Cargar un archivo de configuración del grafo.
2. Ejecutar el mejor algoritmo que usted proponga (indique cuál es) para este problema en dicha configuración y retorne el valor del corte junto con el tiempo de ejecución. La salida que se debe mostrar de ambos métodos de solución es el vector k (secuencia de 0 y 1).
3. Seleccionar un algoritmo específico de los siguientes: algoritmo 2-OPT, BreakOut Search, Scatter Search, Branch and Bound con mejor Heurística para cota inferior. Se ejecuta el algoritmo y se retorna el camino, valor de peso, beneficio y el tiempo de ejecución.
4. Recibir del teclado un camino (secuencia de numeros) e imprimir el valor de peso total y beneficio total.
5. Salir.

El archivo de configuración tiene la siguiente sintaxis. La primera línea contiene tres valores, n , m y W , que corresponden a la cantidad de vértices y de aristas. Después viene una lista de aristas $i j$ con su peso asociado w_{ij} y beneficio p_{ij} ; es decir, 4 valores $i j w_{ij} p_{ij}$. Por ejemplo:

```
4 4 7
0 1 1 4
1 2 2 3
2 3 3 2
3 0 4 1
```

corresponde al siguiente grafo:

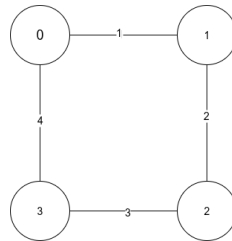


Figure 2: Grafo con 4 vértices

El camino óptimo en este caso es:

```
0 1 2 3 4
peso total: 6
beneficio total: 9
tiempo: 1 [ms]
```

Es crucial implementar estructuras de datos eficientes para el acceso rápido, la búsqueda y el cálculo de máximos. Para ello, se dispone de una variedad de estructuras de datos en la librería STL. Su solución debe estar basada en el método de búsqueda local; en concreto, debe implementar obligatoriamente una variación de **Breakout Search** y **Scatter Search** o **GRASP**. Esto significa que debe realizar una modificación que mejore dichos algoritmos. Es decir, una simple búsqueda local 2-OPT no es suficiente para explorar de manera más eficiente el enorme espacio de combinaciones; por lo tanto, se requieren algoritmos más avanzados.

El entregable debe contener:

- Un makefile que compila el programa principal y los programas de test con compilación separada.
- Un test por cada clase generada. Debe comprobar que la clase funcione correctamente.
- Cada clase está representada por 2 archivos (.cpp) y (.h) y debe ser compilada en forma separada.
- Un main.cpp con un menú.
- pdf del informe

- varios ejemplos de entrada.

Debe entregar una carpeta comprimida cuyo nombre sea ApellidoNombreRUT.zip (o con otra compresión). Esta carpeta contiene archivos Header (.h), clases (.cpp), programas principales (main.cpp y test_XX.cpp), al menos un test por clase y un makefile con compilación separada. Toda clase debe implementarse por separado en dos archivos (.h y .cpp). El nombre de una clase siempre comienza con mayúscula y el archivo que la implementa tiene el mismo nombre. No se permite definir funciones fuera de las clases y cada clase debe servir a un solo propósito o abstracción.

7 Rúbrica

1. (10%) Informe : contiene una descripción de la estrategia de resolución utilizada. Por qué su implementación es eficiente. Debe explicar por qué su propuesta es una mejor aproximación al óptimo. Esta explicación debe basarse en la experimentación que haya realizado con sus algoritmos. 0 pts si ninguna de las dos preguntas anteriores se responde o no se muestra que trabajó probando diferentes alternativas.
2. (30%) Código y correctitud : se revisa si compila (0 pts si no compila) y si se implementan todas las funcionalidades descritas anteriormente. Debe implementar: **2-OPT, BreakOut Search, Scatter Search, Branch and Bound** con mejor heurística. Debe implementar el algoritmo con el mejor óptimo utilizando el método de **Branch and Bound**, con límite de iteraciones y poda. Debe ejecutar todos los algoritmos en los ejemplos entregados, codificados con la estructura indicada (clases+test+main+makefile), incluyendo compilación separada, si todo está descrito por clases y no hay funciones sueltas, salvo la función main. El makefile debe operar correctamente. El código debe ser comprensible (con comentarios, variables con nombres descriptivos, indentación, sin repeticiones forzadas). Se requiere un test adecuado por clase. El programa se ejecuta sin problemas y efectúa las operaciones que se esperan de la tarea (si no se logra ejecutar en algún caso 0 pts). **Si tiene funciones fuera de las clases, excepto la función main, obtendrá 0 puntos.** Debe funcionar correctamente; es decir, para todos los ejemplos, debe entregar un camino con un beneficio mayor o igual al obtenido por el algoritmo de 2-OPT. No debe caer por un segmentation fault ni por errores en ningún archivo de entrada. Debe utilizar contenedores STL para las estructuras de datos.
3. (50%) Eficiencia computacional y calidad de la búsqueda del valor óptimo (**0 pts si no funciona o no se intenta hacerlo eficiente.**) Se verifica que se haya implementado el problema de manera eficiente. Por eficientes se entiende que las operaciones de búsqueda, inserción, borrado, pop y push sean al menos $O(\log(N))$, basándose en contenedores STL para implementar algoritmos propios. Se revisa la calidad de la mejora para determinar el beneficio óptimo y se comparan los caminos con los algoritmos BreakOut y Scatter. El algoritmo seleccionado como el mejor debe basarse necesariamente en el algoritmo de Branch and Bound (**Si no lo está, se descuenta 25% en este ítem**), debe verificar un valor de gran calidad (mayor beneficio) posible según un tiempo máximo.
4. (10%) Revisión de a pares.
5. Bonus 1.0 pt para el programa que presente un valor de beneficio más alto para los casos difíciles y en un tiempo razonable.

8 Entrega

01 de junio a las 11:55PM entrega que incluye lo anterior y de forma individual. 1 punto por cada día de retraso, considerando la entrega con un plazo posterior al plazo indicado. En la misma semana, se llevará a cabo la revisión por pares. **El archivo entregable es una carpeta con el nombre ApellidoNombreRUT del alumno, y debe estar comprimida.** Por ejemplo, en un archivo zip como ApellidoNombreRUT.zip con los archivos requeridos.

vía Classroom